FAQ-14: Should I Replace My Custom SQL Server Monitoring Scripts with the Dynatrace Extension?

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 14 — Custom SQL Server Scripts vs. the Dynatrace Extension | **Created:** July 2026 | **Last Updated:** 11/13/2026

Overview

Most teams arriving at Dynatrace with an established SQL Server estate bring a homegrown monitoring layer with them: a set of custom T-SQL queries polled by Telegraf, cron jobs, or a scheduler — blocked sessions, database status, agent job outcomes, log space, Always On health, instance status, and usually one or two exotic checks like tempdb version-store pressure. The scripts work, so the natural question is *"can't we just keep them?"*

You can — but in a typical estate the **Microsoft SQL Server extension from Dynatrace Hub covers nearly the entire homegrown monitor set out of the box**, and it delivers three things a script can never produce: topology, first-class Smartscape entities, and Davis analysis that knows which instance, database, or availability group a signal belongs to. This entry maps a typical custom monitor set onto the extension's signals, is honest about the gaps, and lays out the decision — including when keeping Telegraf is the right call.

Table of Contents

1. Short Answer
2. Two Views of a Database: Caller-Side vs Server-Side
3. What the Extension Collects Out of the Box

4. Mapping a Typical Homegrown Monitor Set
5. What You Gain Beyond the Numbers
6. Honest Gaps and Constraints
7. Closing Real Gaps with a Small Custom Extension
8. The Keep-Telegraf Fallback
9. Setup Essentials and Cost
10. Recommended Approach
11. Common Objections

Prerequisites

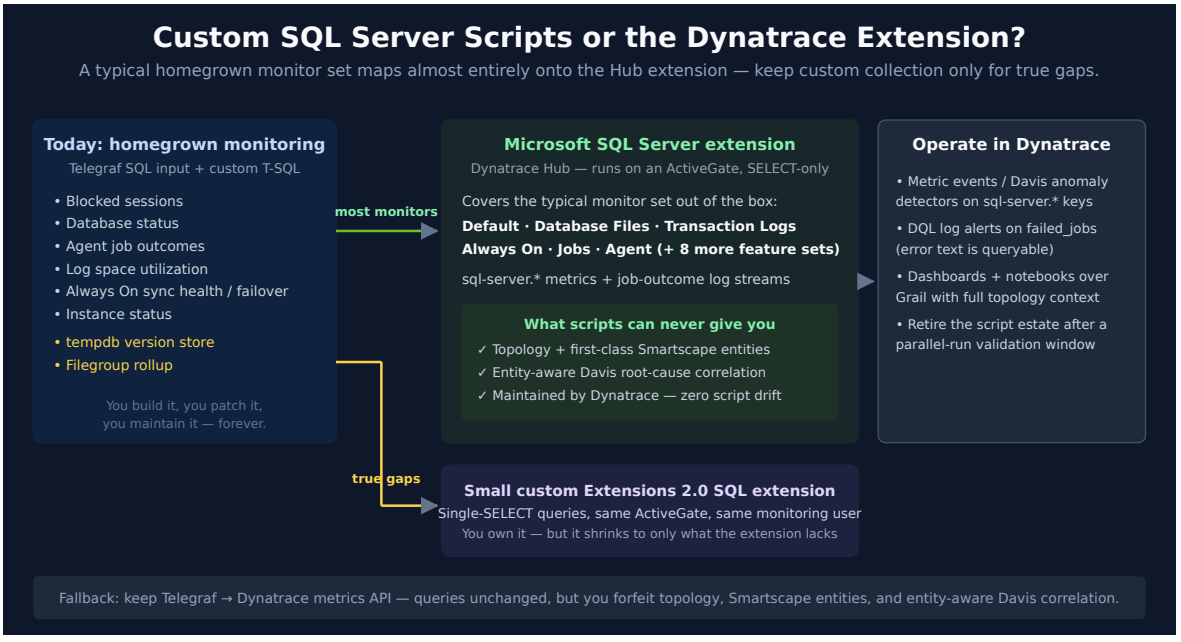
Requirement	Details
Dynatrace Environment	SaaS with Grail
ActiveGate	An ActiveGate group with network reach to the SQL Server estate (host-based; a local OneAgent variant of the extension also exists)
SQL monitoring user	`VIEW SERVER STATE` (SQL Server 2022+: `VIEW SERVER PERFORMANCE STATE`, plus `VIEW SERVER SECURITY STATE` for the Agent feature set's `sys.dm_server_services`); `VIEW ANY DEFINITION` for Always On views; read on `msdb` for jobs/backups; `ALTER ANY DATABASE` (or `CREATE DATABASE` in `master`) if OFFLINE databases must be visible — `VIEW ANY DATABASE` shows ONLINE databases only
Audience	DBA teams with existing script/Telegraf-based SQL Server monitoring; platform teams; account teams
Related series	DBMON (database monitoring mechanics, dashboards, alerting), FAQ-09 (metric vs log query economics), AUTOM (extension deployment at scale)

1. Short Answer

Yes, for almost all of a typical script estate. Deploy the Microsoft SQL Server extension (Hub, ActiveGate-based). In community practice, the monitor set most homegrown script

estates carry is blocked sessions, database status, job outcomes, log space utilization, Always On sync health and failover posture, instance status, and file space — and each of those maps onto a documented `sql-server.*` metric or job-outcome log stream (§ 4). Run that mapping against your own script list before treating the verdict as yours. Keep custom collection only for the checks the extension genuinely doesn't make (in practice: tempdb version-store pressure, and filegroup-level rollups if that exact granularity drives an alert), and implement those as a small Extensions 2.0 SQL extension on the same ActiveGate.

The reason to switch is not just parity. Scripts deliver numbers; the extension delivers numbers **attached to entities** — topology, Smartscape SQL Server instances/databases/availability groups, and Davis anomaly detection that can correlate a signal with the rest of the topology. Bypassing the extension and shipping raw script output into Dynatrace keeps the baselines (custom metrics can use them too) but forfeits the entity context most customers bought the platform for.



> **Sources:** Microsoft SQL Server extension (DT docs), SQL Server Monitoring (Dynatrace Hub), Metric events — Dynatrace Classic (DT docs) — baselines for custom metrics, detailed in § 5.

2. Two Views of a Database: Caller-Side vs Server-Side

Dynatrace observes a database from two independent directions, and the distinction is what makes this question worth asking at all:

	Caller-side (OneAgent / OpenTelemetry)	**Server-side (SQL Server extension)**
Where it measures	Inside your application processes	Inside the database engine, via DMV queries from an ActiveGate
What it sees	Every statement your services execute: response time, failure rate, throughput, per-statement breakdown (`db.system == "mssql"` spans)	Engine internals: blocking, memory/buffer pool, transaction logs, Always On replicas, agent jobs, file space
What it can't see	Why the engine is slow (blocking, log pressure, failover), jobs, databases no instrumented app talks to	Individual statement latency as your application experienced it
Agent on the DB host?	No (measures from the caller)	No (remote from ActiveGate) — a local OneAgent variant exists

Homegrown script estates exist because teams historically only had the server-side view to build on. The caller-side view (covered in the DBMON series) comes free with OneAgent instrumentation of your applications; the extension completes the picture on the server side. **The two are complements, not alternatives** — the migration decision is only about who produces the server-side signals: your scripts, or the extension.

> **Sources:** Microsoft SQL Server extension (DT docs), Microsoft SQL Server local extension (DT docs).
>
> **Derived:** the caller-side/server-side framing is a synthesis of the extension docs and Dynatrace's span-based database monitoring model.

3. What the Extension Collects Out of the Box

The extension organizes collection into **14 feature sets**. The ones that matter for replacing a typical script estate:

Feature set	Key signals (documented keys)
Default (always on)	`sql-server.general.processesBlocked`, `sql-server.databases.state`, `sql-server.uptime`, `sql-server.general.userConnections`, memory/CPU/worker-thread metrics
Transaction Logs	`sql-server.databases.log.percentUsed`, `.log.filesUsedSize`, `.log.filesSize`, growth/shrink/truncation/flush-wait counts
Database Files	`sql-server.databases.file.size` / `.usedSpace` / `.emptySpace` + a `largest_files` log stream (top 100 files every 5 minutes). `usedSpace` and `emptySpace` are reported **only for the database the extension is connected to**
Always On	Availability-group, replica, and database-level health: `sql-server.always-on.ag.synchronizationHealth`, `sql-server.always-on.ar.role`, `sql-server.always-on.ar.failoverMode`, `sql-server.always-on.ar.operationalState`, `sql-server.always-on.db.synchronizationState`, `sql-server.always-on.db.logSendQueueSize` / `.logSendRate` / `.redoQueueSize` / `.redoRate`
Jobs	`current_jobs` log stream — a snapshot of current jobs every 5 minutes (`job_name`, `job_status`, `last_run_outcome`, duration, execution dates) — + `failed_jobs` log stream (`step_name`, `outcome`, `sql_severity`, `retries_attempted`, **error message text** in `content`)
Agent	`sql-server.sql.agent.status`
Locks and waits	`all_requests` log stream — active requests with their locks and waits, every minute

Feature set	Key signals (documented keys)
Memory / Locks / Latches / Queries / Sessions / Replication / Backups	Buffer pool and page life expectancy, deadlocks, latch waits, top longest queries (2016+), session counts, backup age/size

Two structural points worth internalizing:

- **The metric namespace is `sql-server.*`** — these are documented, versioned keys maintained by Dynatrace, not names you invented in a Telegraf config.
- **Job outcomes arrive as log streams, not metrics.** That is an upgrade: the failure message text lands in Grail and is queryable with DQL, which polling `sysjobhistory` into a numeric metric never gave you.

> **Sources:** Microsoft SQL Server extension (DT docs) — feature sets, metric keys, and log-stream attributes are documented on this page; *"are only reported for the database the extension is currently connected to"*; *"The extension fetches current jobs every five minutes."*

4. Mapping a Typical Homegrown Monitor Set

In community practice, the monitor set below is the one seen most often in script/Telegraf estates — your inventory may differ, so run the mapping against your actual script list before deciding. Each coverage cell is read from the extension's documented metric and log-stream list. If your set looks like this, the extension replaces it nearly one-for-one:

Typical homegrown monitor	Extension coverage	Notes
Blocked sessions (poll `sysprocesses`)	✅ `sql-server.general.processesBlocked`	Semantics differ slightly: scripts often count *blocking* SPIDs, the metric counts *blocked* processes — directionally equivalent, don't copy thresholds 1:1
Database status (`sys.databases`)	✅ `sql-server.databases.state`	Per database
Agent job last-run outcome (`msdb.dbo.sysjobhistory`)	✅ `current_jobs` / `failed_jobs` log streams	Richer than the script — error text queryable
Transaction log space	✅ `sql-server.databases.log.percentUsed`	Exact equivalent
Always On sync health	✅ `sql-server.always-on.ag.synchronizationHealth` / `.ar.synchronizationHealth` / `.db.synchronizationHealth`	AG, replica, and database granularity
Failover posture / replica role	✅ `sql-server.always-on.ar.role`, `.ar.failoverMode`, `.ar.operationalState`	Point the Always On config at the **primary** replica
Instance status / uptime	✅ `sql-server.uptime` + connection/worker metrics	Extension connectivity doubles as reachability signal

Typical homegrown monitor	Extension coverage	Notes
Database file space	⚠️ Database Files feature set	File size per database and per file; used/empty space only for the database the extension is connected to
tempdb version-store pressure	❌ Not collected	The one recurring hard gap — see section 7
Filegroup-level %-of-maxsize rollup	⚠️ File-level only	No filegroup dimension exists on the file metrics

> **Sources:** Microsoft SQL Server extension (DT docs).

5. What You Gain Beyond the Numbers

If the extension only matched your scripts metric-for-metric, the migration would be a wash. The case for switching is what arrives *around* the numbers:

1. **Topology and entity context.** Extension signals attach to SQL Server instance/database entities rather than floating as anonymous custom metrics. Since Semantic Dictionary 1.340, Smartscape models SQL Server natively — availability databases, groups, replicas, instances, and databases as first-class typed entities with relationships. A blocked-session spike is one click away from the services calling that database.
2. **Entity-aware Davis analysis.** Every extension metric is eligible for Davis anomaly detection — auto-adaptive and seasonal baselines instead of static thresholds. Your Telegraf metrics can use the same baselines, but they alert as series attached at most to a host — no SQL Server instance, database or availability-group entity — so Davis cannot place them in the database topology when it correlates a problem.

3. **Zero maintenance.** Dynatrace versions the extension against SQL Server releases, DMV changes, and permission-model changes (e.g., the 2022 `VIEW SERVER PERFORMANCE STATE` split). Every one of those is currently your job.
4. **Unified alerting surface.** Metric events, Davis problems, and DQL log alerts on `failed_jobs` all flow into the same problem/notification pipeline as the rest of your observability — no parallel alert path to operate.

In community practice, the maintenance point is the one that lands hardest with DBA teams: the scripts are rarely the problem — the person who wrote them leaving is.

> **Sources:** Microsoft SQL Server extension (DT docs), Semantic Dictionary 1.340 changelog (DT docs) — 19 first-class Smartscape database models incl. SQL Server availability database/group/replica, instance, database; Metric events — Dynatrace Classic (DT docs) — *"Auto-adaptive thresholds and seasonal baselining are available only for metric selector events."* The page is labeled Dynatrace Classic and says *"We encourage you to try Anomaly Detection for more advanced configuration options"* (re-read 10/15/2026); Extend metrics (DT docs) — custom metrics are *"assigned to a host either automatically or by adding the dt.entity.host dimension"*, and *"You can also create custom alerts based on the ingested metrics."* **Derived:** the four-point value framing combines the cited sources with the DPS alerting model.

6. Honest Gaps and Constraints

The migration case survives honesty; hiding these helps no one:

- **No tempdb / version-store metrics.** Nothing in any feature set covers `version_store_reserved_page_count` or the longest-running-transaction counter. If you run read-committed snapshot isolation at scale, this check matters and stays custom.
- **No filegroup dimension.** File metrics are per-database and per-file. A filegroup %-of-maxsize rollup cannot be reconstructed from the documented dimensions.
- **Jobs bill as log ingest,** not metrics (as do the largest-files and top-queries streams). Factor it into DPS planning — volumes are usually small (failed jobs and largest files are capped at 100 records per 5 minutes), but current jobs scale with your job count, and the *Locks and waits* `all_requests` stream emits every active request each minute — size it before enabling. It is also a different meter.

- **Always On needs deliberate config.** Two monitoring configurations: Always On feature set → primary replicas only; everything else → all instances. Connecting primaries *and* secondaries with Always On enabled produces duplicate metrics — Microsoft SQL Server extension (DT docs): *"Don't enable the "Always On" feature set on both the primary and the secondary replica in the same cluster."*
- **Blocked-process semantics** differ from typical scripts (blocked vs blocking counts) — revalidate thresholds during the parallel run.
- **The extension accepts no user-defined SQL.** Custom checks don't bolt onto it; they live in a separate small extension (next section).

> **Sources:** Microsoft SQL Server extension (DT docs) — feature-set catalog, dimension lists, Always On configuration guidance, and DDU/DPS notes, incl. *"top 100 failed jobs * 12 * avg log size"*, *"Number of currently enabled jobs * 12 * avg log size"* and *"avg Number of active requests * 60 * avg log size"*.

7. Closing Real Gaps with a Small Custom Extension

For the checks the extension genuinely lacks, build a **declarative Extensions 2.0 extension with the `sqlServer` data source**. It runs on the same ActiveGate, uses the same monitoring user, and emits gauge/count metrics with dimensions — no agent, no scheduler, no script host.

Constraints that will affect scripts as currently written:

- **Single, plain `SELECT` statements only.** `DECLARE`, table variables, `INSERT…EXEC`, `sp_MSforeachdb`, and `IF/ELSE` are rejected — most homegrown checks need a rewrite into one SELECT (usually straightforward: subqueries + `CROSS JOIN` replace the procedural scaffolding).
- **Comments inside queries are rejected.**
- Default 10-second query timeout, and *"Only one query can be executed at a time"* — another reason a multi-statement script has to collapse into one SELECT.

In community practice, the design rule worth adopting during the rewrite is **emit raw values, not verdicts.** A script that returns `1` when version store exceeds 30% of tempdb bakes the threshold into the collector. Emit the percentage as a gauge and put the `> 30`

in a Dynatrace metric event — thresholds become tunable, and Davis can baseline the raw signal.

The DBMON series carries the mechanics of the server-side view; the SQL data source reference (DT docs) documents the YAML shape, scheduling (per-minute to cron), and dimension model.

> **Sources:** SQL data source reference (DT docs) — *"Only one query can be executed at a time"*, *"Queries containing comments are rejected"*, *"Timeouts are specified in seconds; the default value is 10."*

8. The Keep-Telegraf Fallback

Keeping Telegraf and pointing it at Dynatrace is legitimate — the Telegraf output plugin ships to the local OneAgent metric API (`:14499/metrics/ingest`) or the Metrics API v2, and the queries stay exactly as written since Telegraf executes them itself.

Be clear-eyed about what that choice costs:

	Extension path	**Keep-Telegraf path**
Query rewrite	Gaps only (single-SELECT)	None
Topology / Smartscape entities	✅	❌ plain custom metrics
Davis baselines	✅ entity-aware — correlated into problems	⚠️ available on the series, but no SQL Server entity context
Maintenance owner	Dynatrace (+ your small gap extension)	You, indefinitely
Agents to operate	ActiveGate (already required)	ActiveGate **+** Telegraf fleet

If you skip the extension entirely and bring everything in through Telegraf, you are running Dynatrace as a metrics bucket — forfeiting the topology, entity model, and entity-aware Davis correlation that differentiate it, and handicapping the platform's ability to deliver the visibility and predictability you adopted it for. The fallback earns its keep as a **transition**

state (parallel-run during migration) or for the rare check that resists a single-SELECT rewrite — not as the destination.

> **Sources:** Microsoft SQL Server extension (DT docs). Metric events — Dynatrace Classic (DT docs) — auto-adaptive and seasonal baselines for metric selector events. **Derived:** the trade-off table combines the extension's documented entity model with the plain-custom-metric behavior of API-ingested series.

9. Setup Essentials and Cost

1. Install the extension from **Dynatrace Hub** (remote flavor; a local OneAgent variant exists for hosts already running OneAgent).
2. Designate an **ActiveGate group** for DB connections — each monitoring configuration handles hundreds of active endpoints on a single 2 vCPU / 4 GiB ActiveGate, with automatic failover within the group.
3. Create the **monitoring SQL user** (grants in Prerequisites). The extension only executes `SELECT`s.
4. Enable feature sets to match your monitor inventory — Default, Database Files, Transaction Logs, Always On, Jobs, Agent covers the typical set.
5. Set up the **two Always On configurations** (section 6).
6. **Cost:** the extension is free; you pay for ingested data — `sql-server.*` keys as metrics ingest, the jobs/files/queries streams as log ingest.

Verification once data flows — blocked processes from the Default feature set:

> **Sources:** Microsoft SQL Server extension (DT docs) — Hub install, ActiveGate-group placement and failover, monitoring-user grants, and the feature-set list, Microsoft SQL Server local extension (DT docs), SQL data source reference (DT docs) — the SQL data source executes queries only. The sizing figure is published: *"Each monitoring configuration can handle hundreds of active endpoints simultaneously on a single ActiveGate with 2vCPU and 4GiB RAM."*; the failed-jobs query below uses the documented `failed_jobs` fields (*"job_name, step_name, outcome, content, duration, instance, server, sql_severity, retries_attempted, start_execution_date, stop_execution_date"*) and *"The extension fetches failed jobs every five minutes."*

```
// Blocked processes from the SQL Server extension (Default feature set)
timeseries blocked = avg(`sql-server.general.processesBlocked`), from:-24h
```

And the failed-jobs log stream — the failure message text is right there in Grail. Two details matter. Filter on the extension and the `failed_jobs` event group, the way the extension documentation does, rather than on `last_run_outcome`: that field is on the `current_jobs` stream, a snapshot taken every 5 minutes, so counting its records counts snapshots rather than failures. And count distinct `stop_execution_date` values, so a failure that appears in more than one 5-minute fetch is counted once. **This query follows the documented field list but has not been run against live SQL Server data** (the validation tenant has no SQL Server extension deployed; it executes and returns no rows) — confirm the counts on your own estate before alerting on them. Extension 3.1.2 and later route its logs through dedicated OpenPipeline pipelines, and the extension notes that *"OpenPipeline also handles some field names differently, for example, capitalization"* — check `job_name` and `stop_execution_date` on one live record first:

```
// Failed SQL Server Agent jobs in the last 24h (Jobs feature set, failed_jobs
stream).
// Not yet run against live SQL Server data — verify on your estate.
fetch logs, from:-24h
| filter dt.extension.name == "com.dynatrace.extension.sql-server" and
event.group == "failed_jobs"
| summarize {failures = countDistinct(stop_execution_date), latest =
max(stop_execution_date)}, by:{job_name, server}
| sort failures desc
```

10. Recommended Approach

The sequence below is community practice built on the two pages cited; each step's mechanics are sourced in the section it points to.

1. **Inventory your script estate** — every query, what it measures, and which alerts consume it.
2. **Map each monitor** against section 4. Expect the large majority to land on documented `sql-server.*` signals.
3. **Deploy the extension** per section 9; run it **in parallel** with the scripts for one or two operational cycles.
4. **Rebuild alerts on the Dynatrace side** — metric events / Davis anomaly detectors on the metrics, a DQL log alert on `failed_jobs` — revalidating each threshold against

the extension's semantics rather than copying script thresholds blind.

5. **Rewrite only the true gaps** as a small EF 2.0 SQL extension (single-SELECT, raw values, thresholds in metric events).
6. **Retire the redundant scripts** after the parallel run confirms parity.
7. Reserve keep-Telegraf for transition or for checks that genuinely resist rewrite — not as the standing architecture.

> **Sources:** Microsoft SQL Server extension (DT docs), SQL data source reference (DT docs).

11. Common Objections

Objection	Response
"Our scripts already work."	They work at the cost of permanent maintenance and zero context. The extension delivers the same signals with topology, Smartscape entities, and entity-aware Davis correlation — and Dynatrace maintains it.
"We have checks the extension doesn't cover."	Usually one or two (tempdb version store is the classic). Those shrink into a small EF 2.0 extension; they don't justify keeping the whole estate.
"We don't want an agent on the DB hosts."	The extension is remote — it runs on an ActiveGate and queries over the network. Nothing installs on the database host.
"Our thresholds are tuned; we can't lose them."	Keep them as metric events during the parallel run, then let Davis baselines take over where they prove better. Note the blocked-sessions semantic difference before copying values.
"The DBA team owns monitoring, not the observability team."	The monitoring SQL user, feature-set selection, and gap-extension queries all remain DBA-owned artifacts — the platform team owns the ActiveGate. The split is cleaner than a shared Telegraf config.
"Isn't this just more ingest cost?"	The extension itself is free; you pay for data — and you were already paying (or about to) to ingest the Telegraf versions of the same signals, without the entity context.

> **Sources:** Microsoft SQL Server extension (DT docs) — the remote-collection model behind the *no agent on the DB host* answer, and the covered feature sets, SQL Server Monitoring (Dynatrace Hub) — publisher and support route. **Derived:** the objection/response pairings are this entry's framing of the trade-offs; no source argues them.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.